

# Logic Design and Switching Theory

## LECTURE – 10

---

Subtitle: Introduction to SLC, Latches and  
FlipFlops

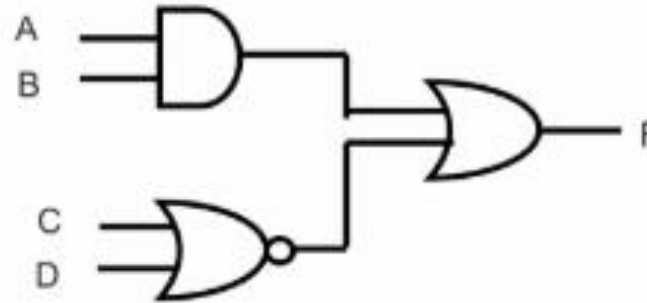
By

Engr. Dr. Hera noor

# Sequential vs. Combinational

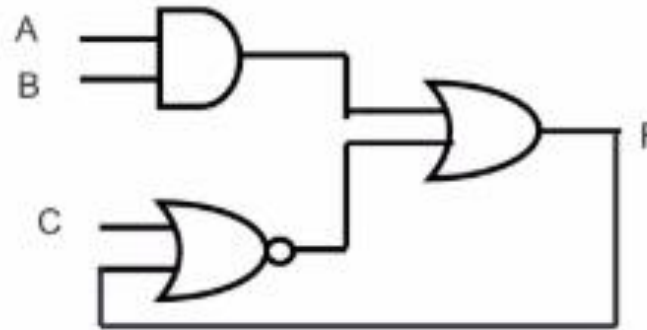
## Combinational Circuit

- output determined solely by inputs



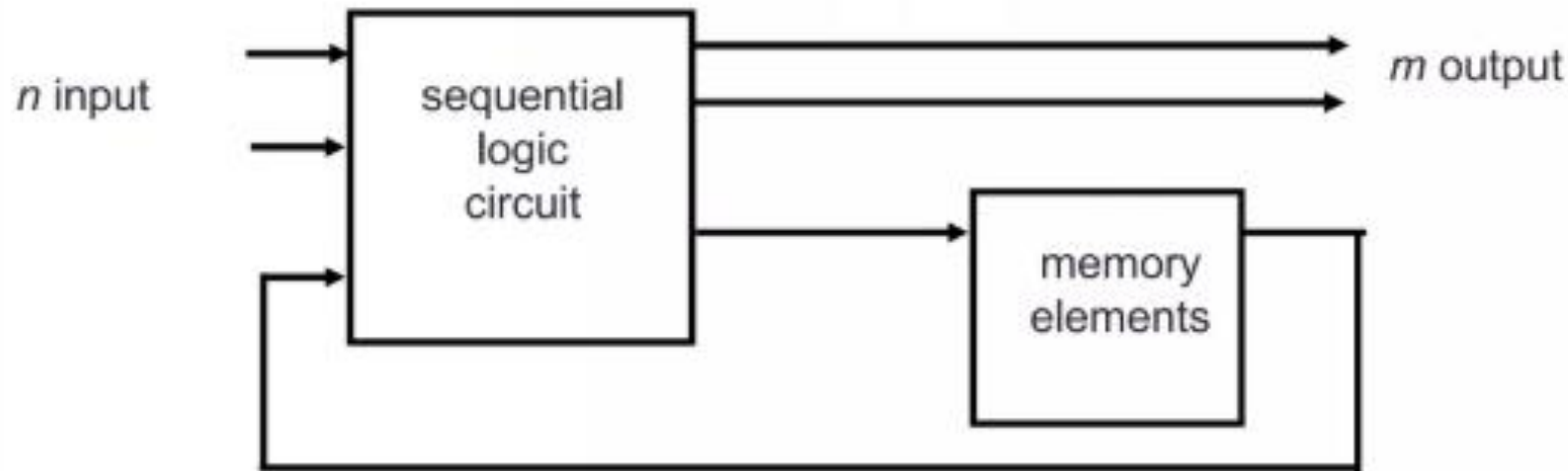
## Sequential Circuit

- output determined by inputs AND previous outputs

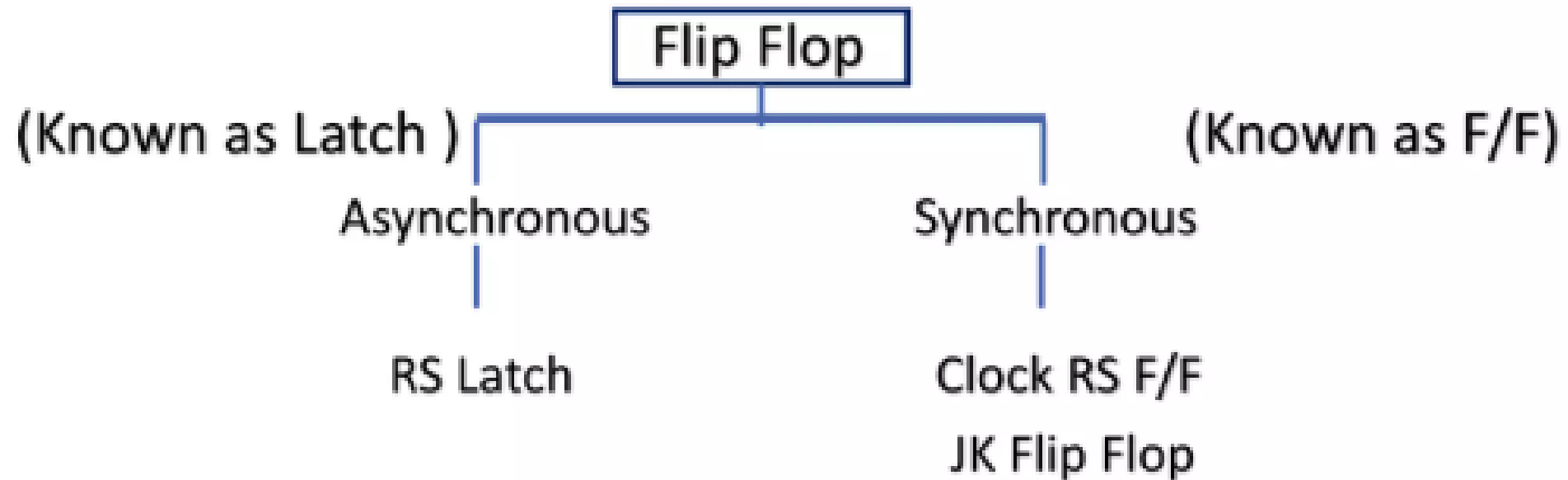


# Sequential Circuits

- Sequential Logic Circuit contains logic gates arranged in parallel and its output is not only determined by the combination of the current input, but also the prior output.
- The circuit also contains **memory elements** that enable it to store the information of the prior output.



# Asynchronous Vs. Synchronous



# Synchronous Sequential Circuits

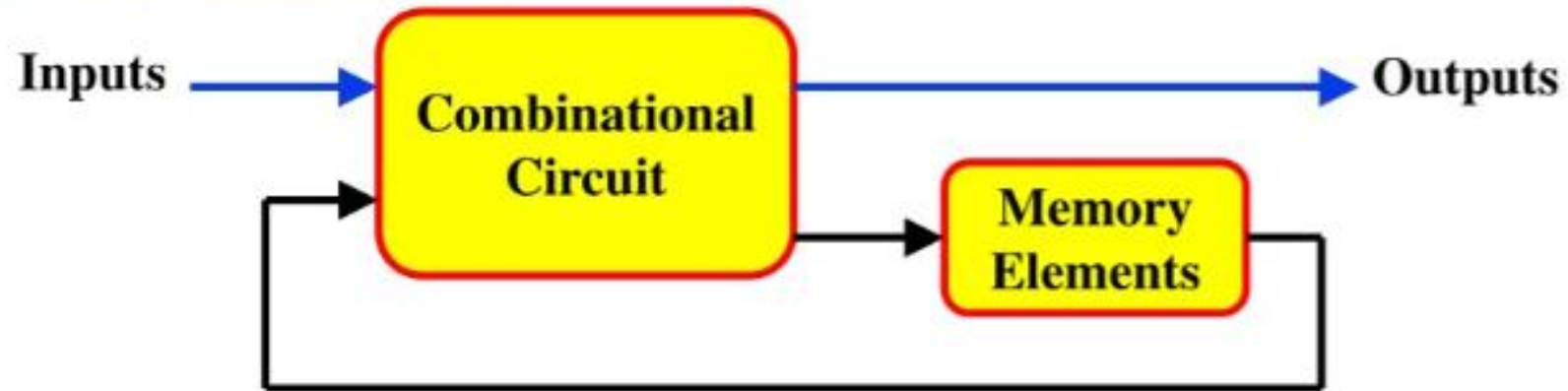
- Circuit output changes only at some discrete instants of time.
- The circuit achieves synchronization by using a timing signal called as the *clock*.
- A clock is used for synchronization
  - ✓ Memory elements are affected only with the arrival of a clock pulse
  - ✓ If memory elements use clock pulses in their inputs, the circuit is called
    - Clocked sequential circuit (FlipFlops)
- The status of memory element is effected only at the active edge of the clock, if input is changed.
- Slower because of clocking.

# Asynchronous Sequential Circuits

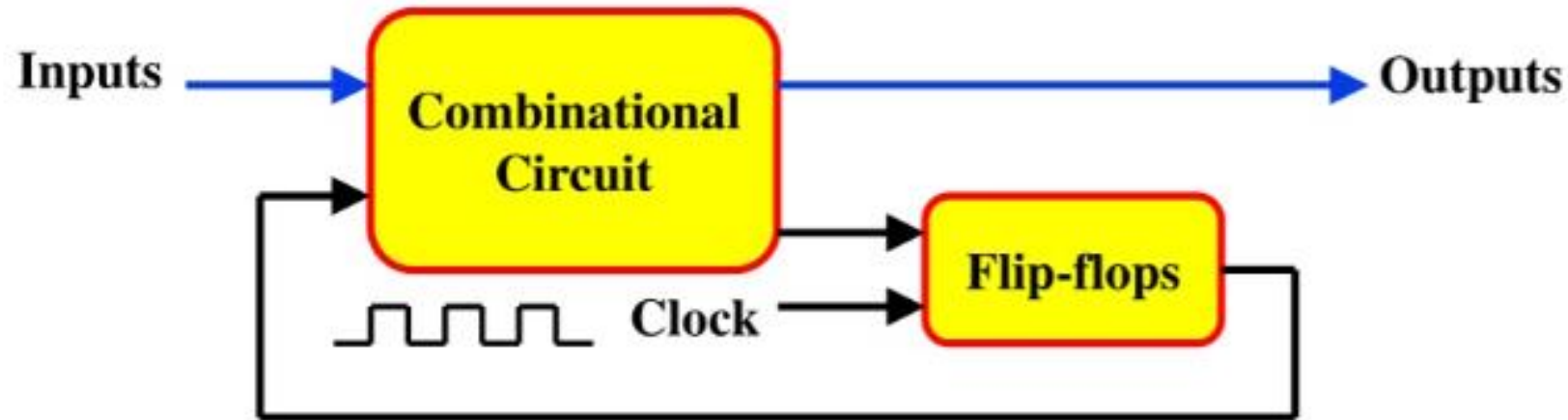
- Circuit output can change at any time (clock less).
- Unclocked latch is used as a memory element.
- Faster because of no clock.
- Status of memory element is changed at any time as soon as input is changed.

# Asynchronous Vs. Synchronous

## □ Asynchronous



## □ Synchronous



# Memory Element

**Memory elements are digital devices that can store one bit of information — either a 0 or a 1. They hold the state of the system and update it only at specific times, usually synchronized by a clock signal.**

**Latches**

**Flipflops**

**Clock**



# Memory Element

## Latches:

- Basic memory elements that can store a bit.
- Level-sensitive (they respond to the level of the control signal).
- Common types: SR (Set-Reset) latch, D (Data) latch.
- Example: SR Latch can be built using two cross-coupled NOR or NAND gates.

## Flip-Flops:

- More controlled than latches; they are edge-triggered (respond to the rising or falling edge of the clock signal).

### Types:

- SR Flip-Flop – similar to SR latch but clocked.
- D Flip-Flop – stores the input (D) on the clock edge.
- JK Flip-Flop – versatile; can toggle, set, or reset.
- T Flip-Flop – toggles state when input is 1 on clock edge.

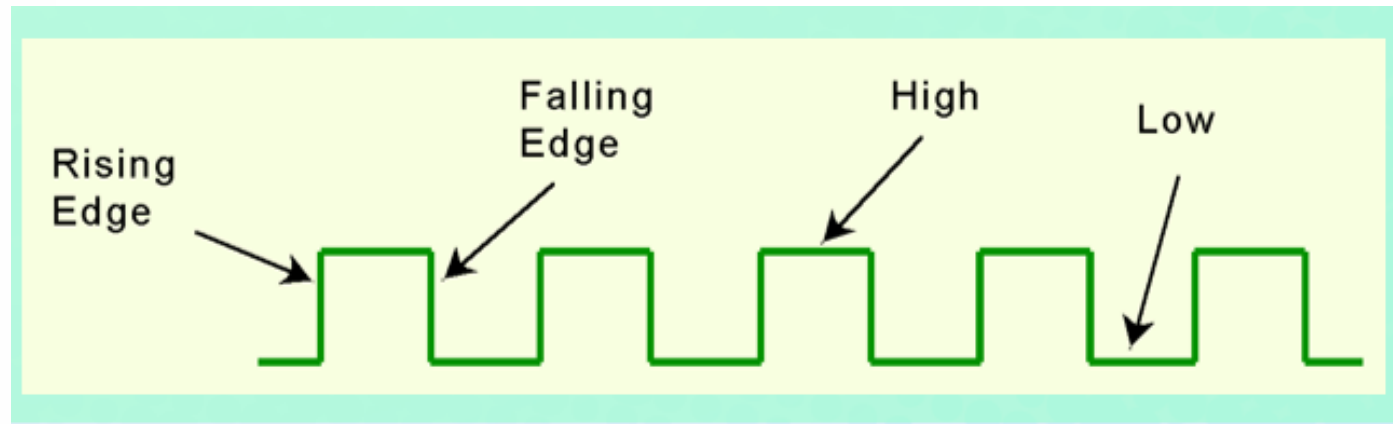
# Clock

Most sequential circuits are clocked, meaning they update their states on clock edges (positive or negative). This:

- ✓ Prevents race conditions
- ✓ Controls the flow of data
- ✓ Ensures all flip-flops change state simultaneously

**It emits a series of pulses with a precise pulse width and precise interval between consecutive pulses**

**Timing interval between the corresponding edges of two consecutive pulses is known as the clock cycle time, or period**



# Applications of Sequential Circuits

- ✓ **Registers (temporary data storage)**
- ✓ **Counters (frequency dividers, event counting)**
- ✓ **Finite State Machines (FSMs in vending machines, traffic lights)**
- ✓ **Memory devices**
- ✓ **Control units in CPUs**

# Memory Element

## Latches:

- Basic memory elements that can store a bit.
- Level-sensitive (they respond to the level of the control signal).
- Common types: SR (Set-Reset) latch, D (Data) latch.
- Example: SR Latch can be built using two cross-coupled NOR or NAND gates.

## Flip-Flops:

- More controlled than latches; they are edge-triggered (respond to the rising or falling edge of the clock signal).

### Types:

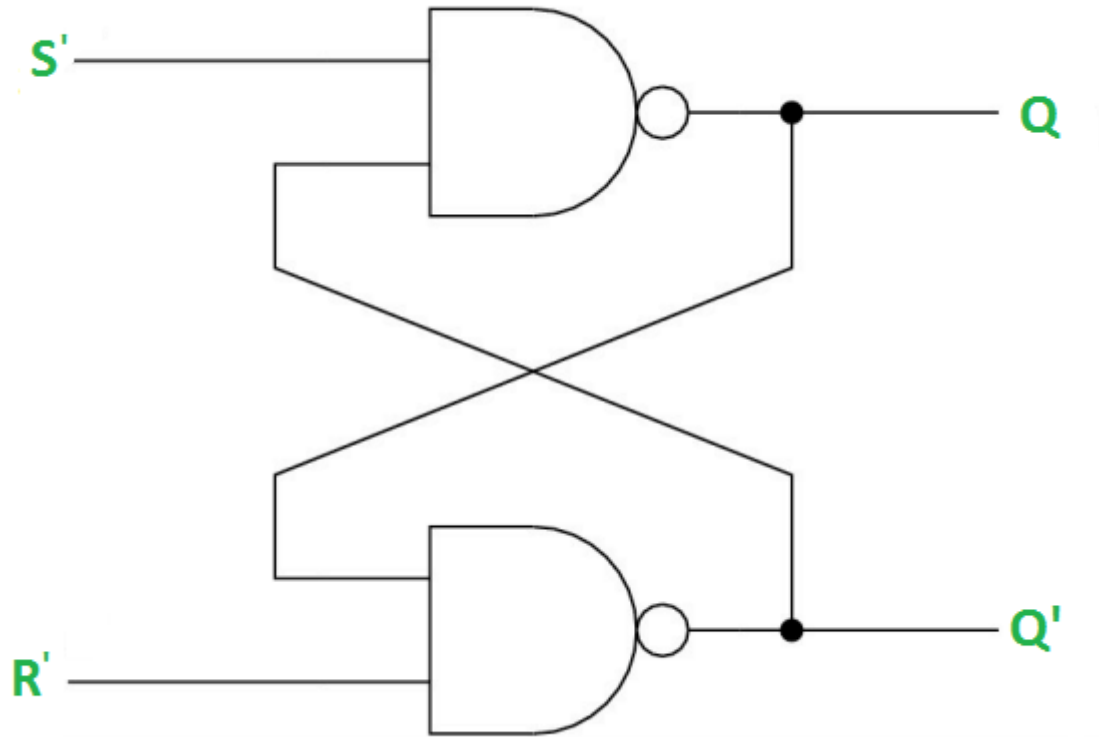
- SR Flip-Flop – similar to SR latch but clocked.
- D Flip-Flop – stores the input (D) on the clock edge.
- JK Flip-Flop – versatile; can toggle, set, or reset.
- T Flip-Flop – toggles state when input is 1 on clock edge.

# SR Latch

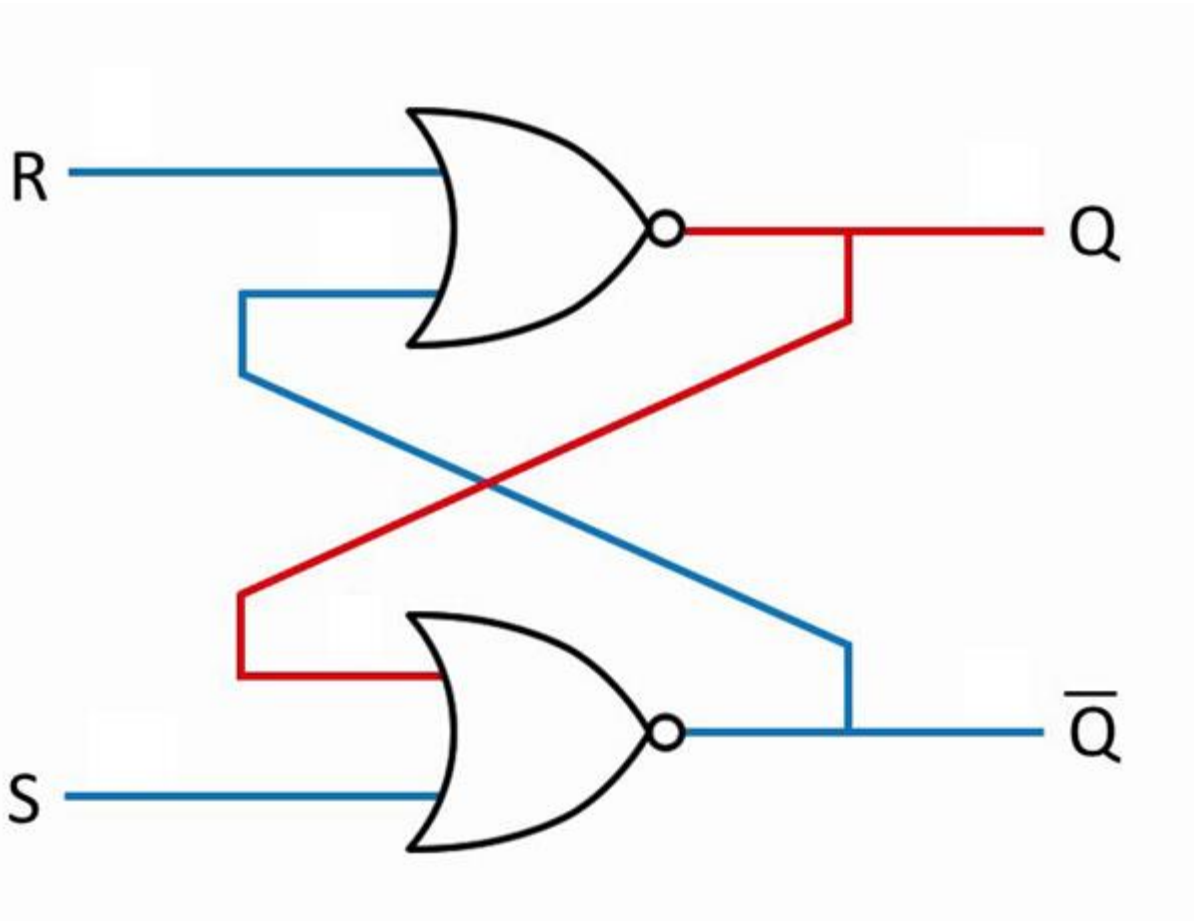
- Made using NOR or NAND gates.
- Has two inputs: Set (S) and Reset (R)
- Two outputs: Q and Q'

| S | R | Q (next state) | Condition       |
|---|---|----------------|-----------------|
| 0 | 0 | Q (no change)  | Hold            |
| 0 | 1 | 0              | Reset           |
| 1 | 0 | 1              | Set             |
| 1 | 1 | Invalid        | Forbidden state |

# SR Latch with NAND gate



# SR Latch with NOR gate



# Flip flops

The flip-flop is a bi-stable device hence they are called **bi-stable multivibrator**.

It exists of two states and in the absence of input, remains in that state. Thus the flip-flop can function as a 1-bit memory.

The flip-flop has two outputs, which are always complements of each other. These are generally labelled as Q and  $\bar{Q}$ .



# Flip flops

- They are also known a **signal change sensitive device**, which means that the change in the level of clock signal bring change in the output of the flipflop.
- They are used to control the digital circuits functionally, as they can change the operation of digital circuits depending on the state.

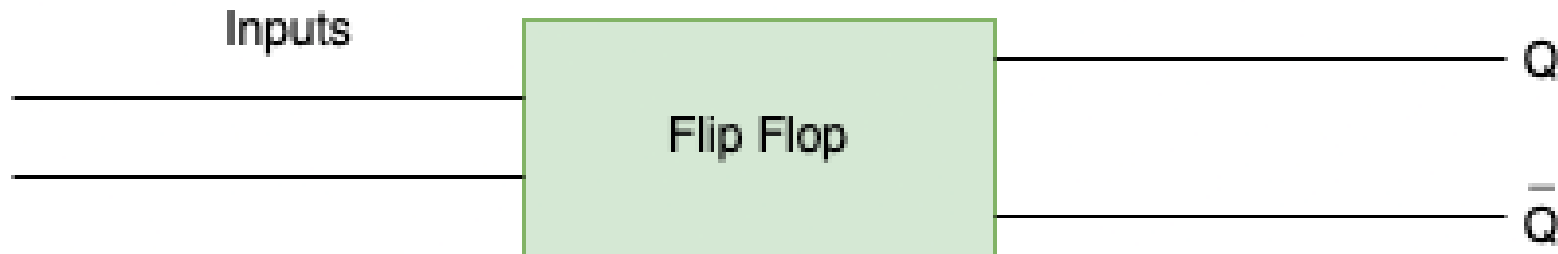
# Flip flops

## Types of flip flop

- SR flip flop
- JK flip flop
- D flip flop
- T flip flop

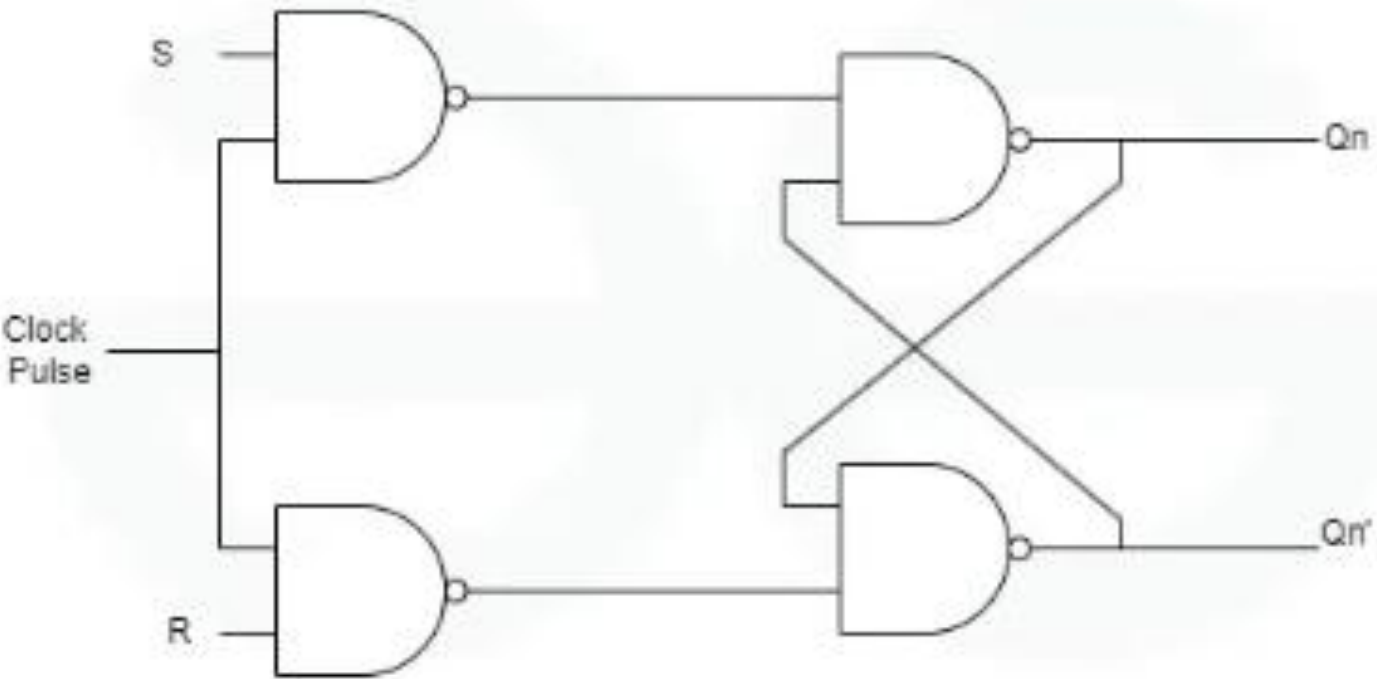
## Applications of Flip flop

- Counters
- Registers
- Logic controller
- Memory



*Diagrammatic Representation of Flip Flop*

# SR Flip flops



# SR Flip flops

- Clock-controlled version of SR Latch.
- Edge-triggered; state changes on clock transition.



# SR Flip flops

|   |    |    |    |    |
|---|----|----|----|----|
|   | 00 | 01 | 11 | 10 |
| 0 | 0  | 1  | 0  | 0  |
| 1 | 1  | 1  | X  | X  |

$$Q_{n+1} = S + Q_n R'$$

(a) Characteristic Table

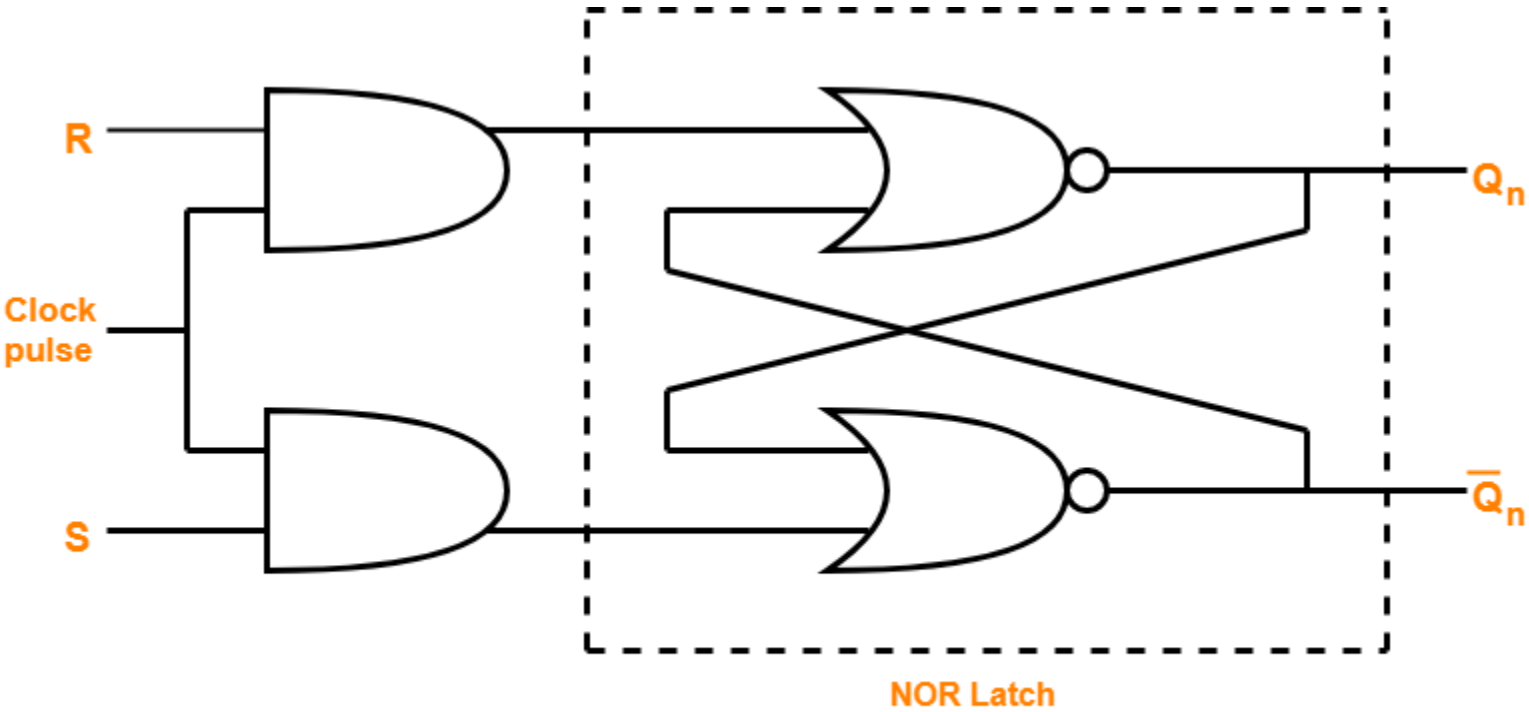
| Current Inputs | Current State | Next State |
|----------------|---------------|------------|
| SR             | $Q_n$         | $Q_{n+1}$  |
| 00             | 0             | 0          |
| 00             | 1             | 1          |
| 01             | 0             | 0          |
| 01             | 1             | 0          |
| 10             | 0             | 1          |
| 10             | 1             | 1          |
| 11             | 0             | —          |
| 11             | 1             | —          |

# SR Flip flops

- **Excitation table**
- The **excitation table** of an SR flip-flop defines the required inputs (S and R) to transition the flip-flop from its current state ( $Q_n$ ) to a desired next state ( $Q_{n+1}$ ). This table is crucial in designing sequential circuits because it tells us what input values are needed to achieve a specific output change.

| $Q_n$ | $Q_{n+1}$ | <b>S</b> | <b>R</b> |
|-------|-----------|----------|----------|
| 0     | 0         | 0        | 0        |
| 0     | 1         | 1        | 0        |
| 1     | 0         | 0        | 1        |
| 1     | 1         | 0        | 0        |

# SR Flip flops



SR Flip Flop Using NOR Latch

# SR Flip flops

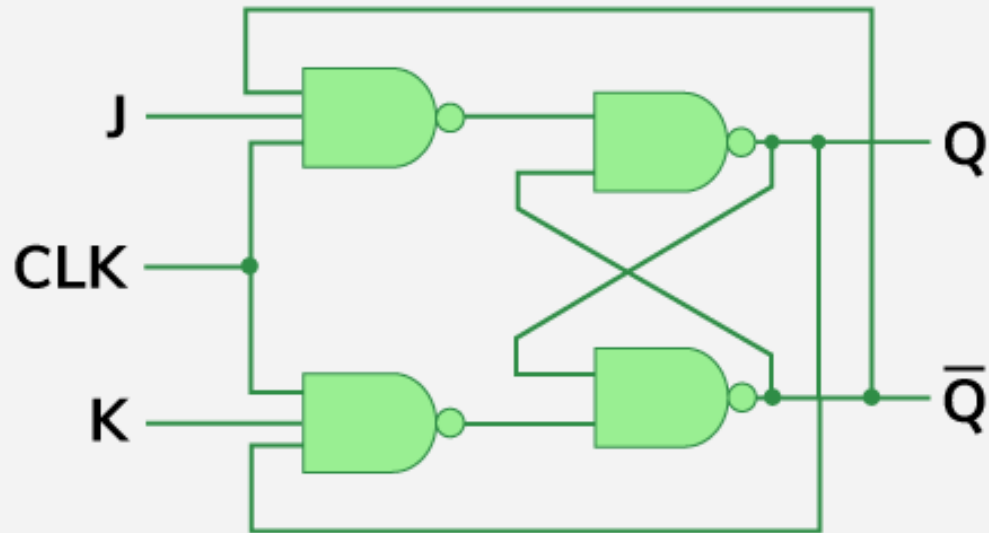
|   |   | BC |    |    |    |
|---|---|----|----|----|----|
|   |   | 00 | 01 | 11 | 10 |
| A | 0 |    |    |    |    |
|   | 1 |    |    |    |    |

$$Q_{n+1} = S + Q_n R'$$

---

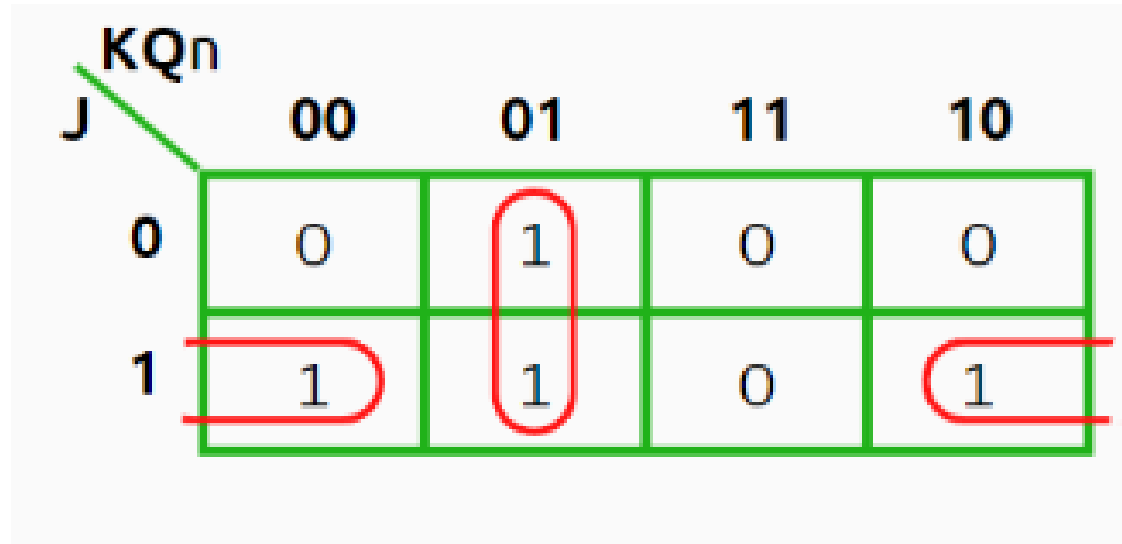


# JK Flip flops



| J | K | $Q_N$ | $Q_{N+1}$ |
|---|---|-------|-----------|
| 0 | 0 | 0     | 0         |
| 0 | 0 | 1     | 1         |
| 0 | 1 | 0     | 0         |
| 0 | 1 | 1     | 0         |
| 1 | 0 | 0     | 1         |
| 1 | 0 | 1     | 1         |
| 1 | 1 | 0     | 1         |
| 1 | 1 | 1     | 0         |

# JK Flip flops



A Karnaugh map for the JK flip-flop next-state equation. The map is a 2x4 grid with J on the vertical axis and KQ<sub>n</sub> on the horizontal axis. The J axis has values 0 and 1. The KQ<sub>n</sub> axis has values 00, 01, 11, and 10. The cells contain the next state value Q<sub>n+1</sub>. Red circles highlight the 1s in the J=0, KQ<sub>n</sub>=01 cell and the 1s in the J=1, KQ<sub>n</sub>=00, 01, and 10 cells. A green line outlines the entire map.

| J \ KQ <sub>n</sub> | 00 | 01 | 11 | 10 |
|---------------------|----|----|----|----|
| 0                   | 0  | 1  | 0  | 0  |
| 1                   | 1  | 1  | 0  | 1  |

$$Q_{n+1} = \bar{K}Q_n + J\bar{Q}_n$$

# JK Flip flops

- Similar to SR but no invalid state.
- When  $J = K = 1$ , it toggles output.

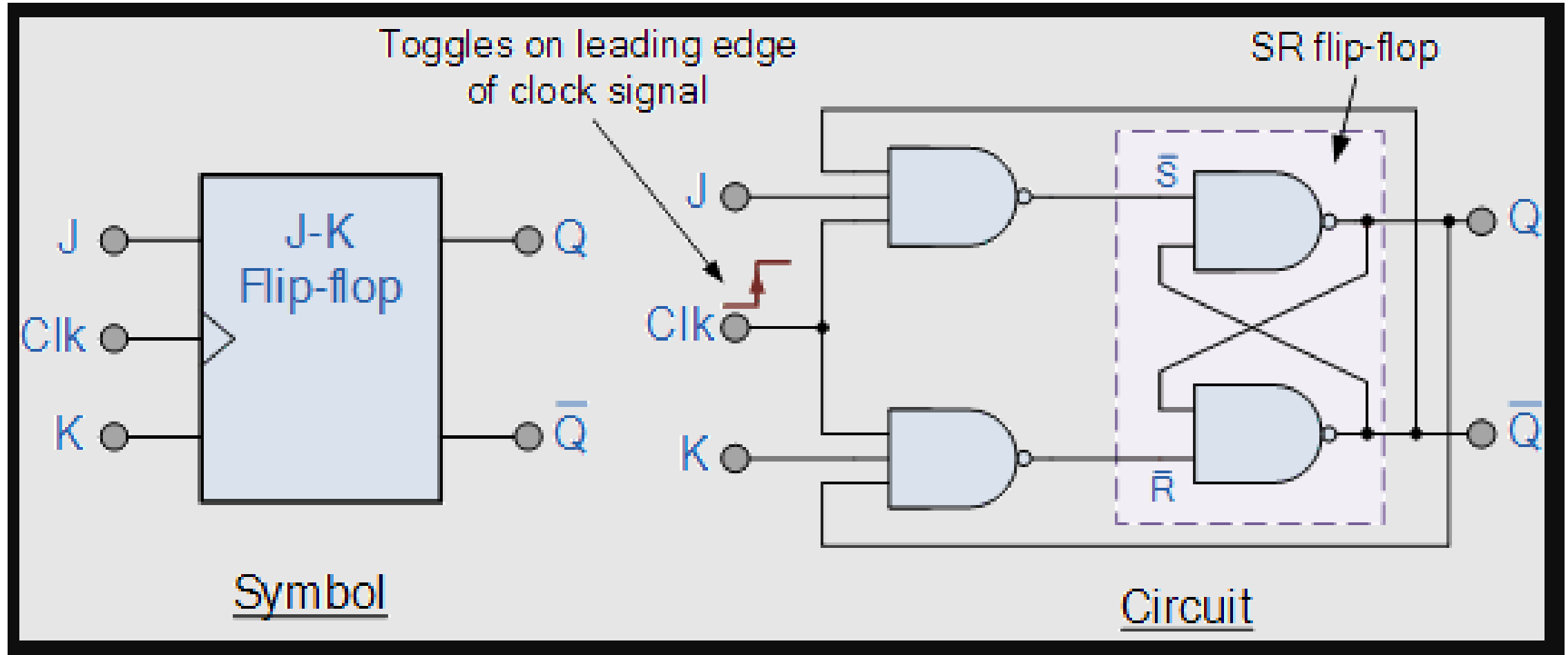
| J | K | Q(next) | Description |
|---|---|---------|-------------|
| 0 | 0 | Q       | No change   |
| 0 | 1 | 0       | Reset       |
| 1 | 0 | 1       | Set         |
| 1 | 1 | Q'      | Toggle      |

# JK Flip flops

- Excitation Table.

| <b>Q(current)</b> | <b>Q(next)</b> | <b>J</b> | <b>K</b> |
|-------------------|----------------|----------|----------|
| 0                 | 0              | 0        | X        |
| 0                 | 1              | 1        | X        |
| 1                 | 0              | X        | 1        |
| 1                 | 1              | X        | 0        |

# JK Flip flops



# Applications of JK Flip-Flop

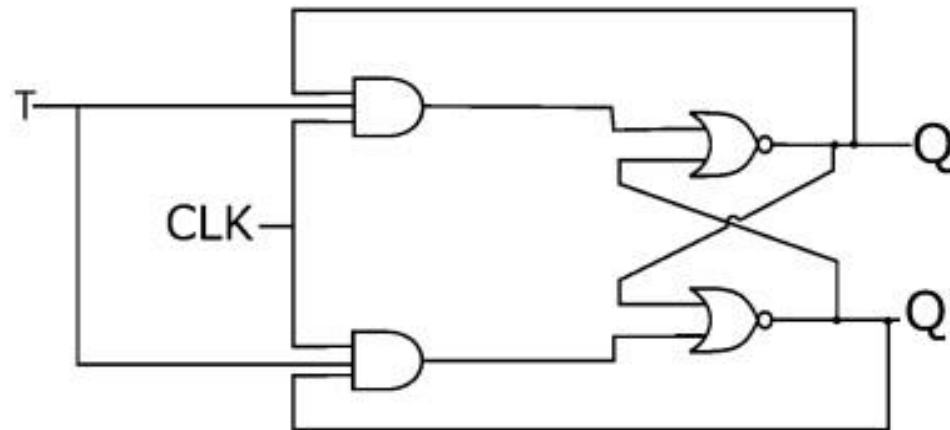
- Counters: Counters are very essential components for the application of frequency dividers and event sequencers where there is a need of storing and propagating the count value. We can design binary synchronous and asynchronous counters using JK-flipflop.
- Shift Registers: For data storage and manipulation, serial-to-parallel or parallel-to-serial data conversion the shift register are widely used. Registers can store and shift the binary data in a sequential manner. We can design it by JK-flipflops.
- Memory Units: JK-flipflop itself act as a memory unit to store binary information. By making a sequential chain of JK-flipflops we can use it even as RAM.

# Advantages vs Disadvantages

- **Versatility:** As discussed above, JK-flipflops can be used as a basic memory element or a primary building block of further complex memory design. It is very much adaptive as it can be operated in both synchronous and asynchronous modes.
- **Toggle Functionality:** The application which are required to get output as its complement of input that also can be developed by JK-flipflops as when  $J=K=1$  it triggers toggle state which gives output which is complement with its each clock pulse.
- **Error Detection and Correction:** We can use a complex circuit built by JK-flipflops which can detect and correct information during data-transmission.
- **Complexity:** Compared to other types of flipflops(D,T, SR), JK flipflop requires additional logic gates to implement which consumes extra memory resources and increases complexity to operate.
- **Propagation Delay:** This is the major problem present in JK-FF. Propagation delay results a timing delay in certain application which are time-flow sensitive.
- **Race Problem:** This issue arises when the clock input's timing pulse isn't given enough time to turn "Off" before the output Q's state is altered.

# T Flip flops

- T flip flop is known as Toggle Flip Flop because it is able to toggle its output depending upon on the input. Toggle basically indicates that the bit will be flipped i.e., either from 1 to 0 or from 0 to 1. Here, a clock pulse is supplied to operate this flip flop, hence it is a clocked flip-flop.
- Toggle flip-flop (T = Toggle)
- When  $T = 1$ , output toggles.





# T Flip flops

TT

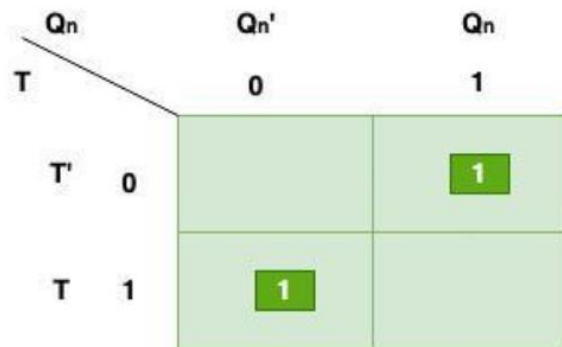
| T | Q(next) |
|---|---------|
| 0 | Q       |
| 1 | Q'      |

CT

| T | Q(next) |
|---|---------|
| 0 | Q       |
| 1 | Q'      |

Excitation table

| Q(current) | Q(next) | T |
|------------|---------|---|
| 0          | 0       | 0 |
| 0          | 1       | 1 |
| 1          | 0       | 1 |
| 1          | 1       | 0 |

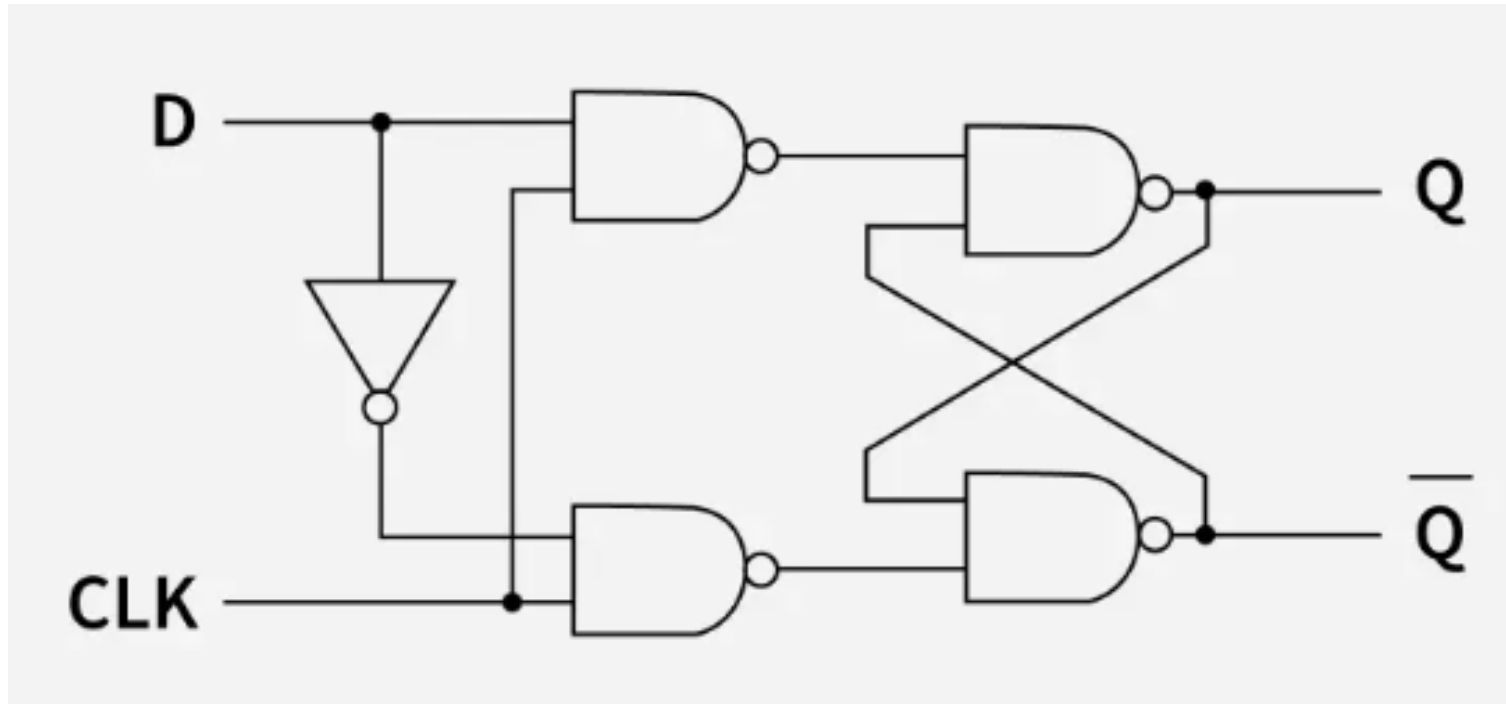


K-Map for T Flip Flop

$$Q(n+1) = TQn' + T'Qn = T \oplus Qn$$

# D Flip flops

- Only one input: **D (Data)**
- Output follows input on clock edge.



# D Flip flops

When the clock signal is high, the flip flop samples and stores D input. The value that was previously fed into the D input is reflected at the flip flop's Q output.

If  $D = 0$  then Q will be 0.

If  $D = 1$  then Q will be 1.

The Q' output of is complemented

If  $Q = 0$  then Q' will be 1.

If  $Q = 1$  then Q' will be 0.

| D | CLK              | $\bar{Q}$ |
|---|------------------|-----------|
| 0 | 1 (Raising Edge) | 0         |
| 1 | 1 (Raising Edge) | 1         |

*Truth Table of D Flip Flop*

# D Flip flops

$$Q(n+1) = D(n)$$

This characteristic equation of D flip flop states **“that the output of the flip flop at the next clock cycle will be equal to the input at the current clock cycle”**.

| D | Q(Current) | Q(n+1)<br>(Next) |
|---|------------|------------------|
| 0 | 0          | 0                |
| 0 | 1          | 0                |
| 1 | 0          | 1                |
| 1 | 1          | 1                |

*Characteristics table of D Flip Flop*

# Advantages of D Flip flops

- The computation speed of D flip flop is very fast compared to other flip flops.
- D flip flop requires very few components to design which makes it simple to understand.
- **Note:** D flip flops are glitch prone. When input varies fast, flip flop output may glitch. Digital circuit glitches are hard to identify and fix.

A "glitch" in a flip-flop refers to a temporary, unwanted change in the output signal that occurs during the flip-flop's switching process.

| Q(n) | D(n) | Q(n+1)<br>(Next) |
|------|------|------------------|
| 0    | 0    | 0                |
| 0    | 1    | 1                |
| 1    | 0    | 0                |
| 1    | 1    | 1                |

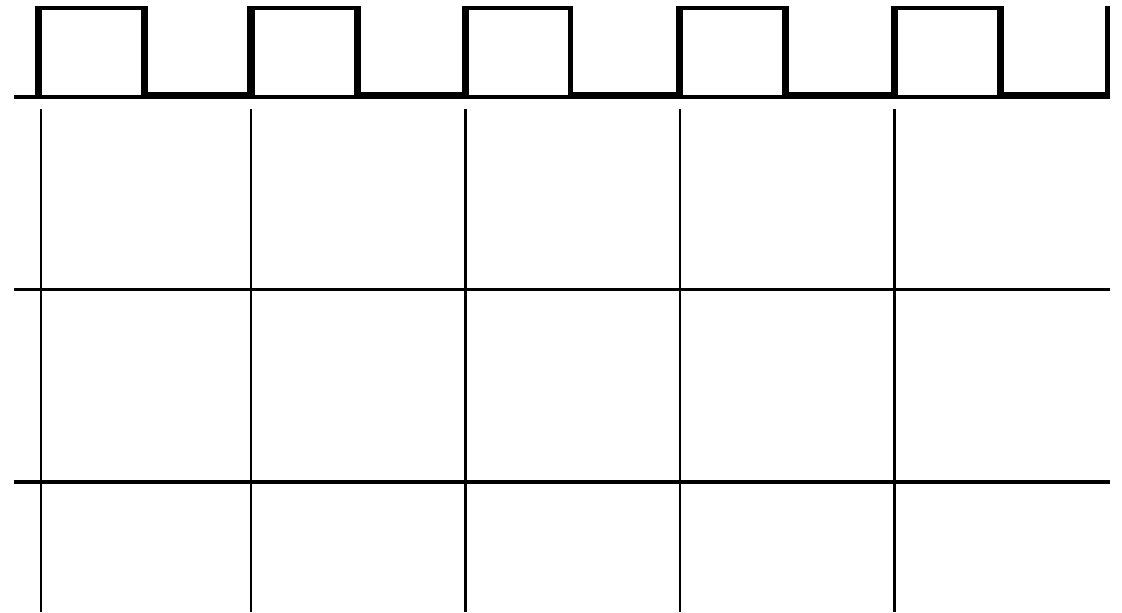
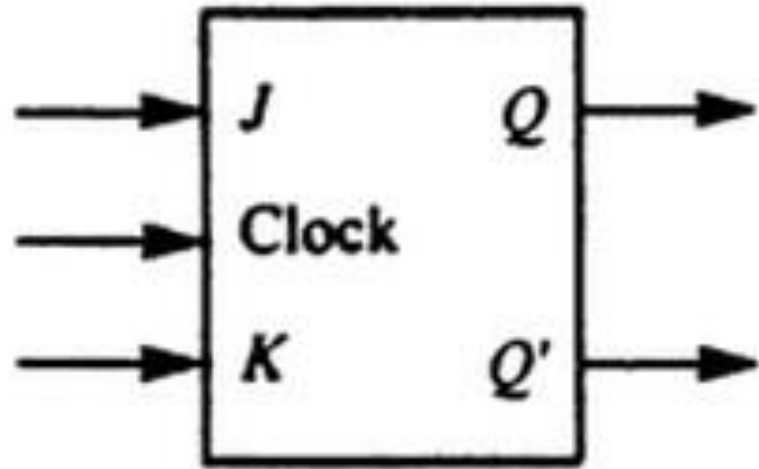
# Race Around Condition

A race around condition is a type of issue that occurs in sequential logic circuits, especially flip-flops like the JK flip-flop, when certain inputs cause the output to toggle multiple times within a single clock cycle. This happens when the circuit doesn't have enough time to settle to a stable state before it's triggered again.

Mostly in:

- JK flip-flops, especially when  $J = K = 1$
- Asynchronous systems where there's no proper clock synchronization
- Systems with propagation delays that allow multiple toggles before the clock pulse ends

# Race Around Condition



# Race Around Condition

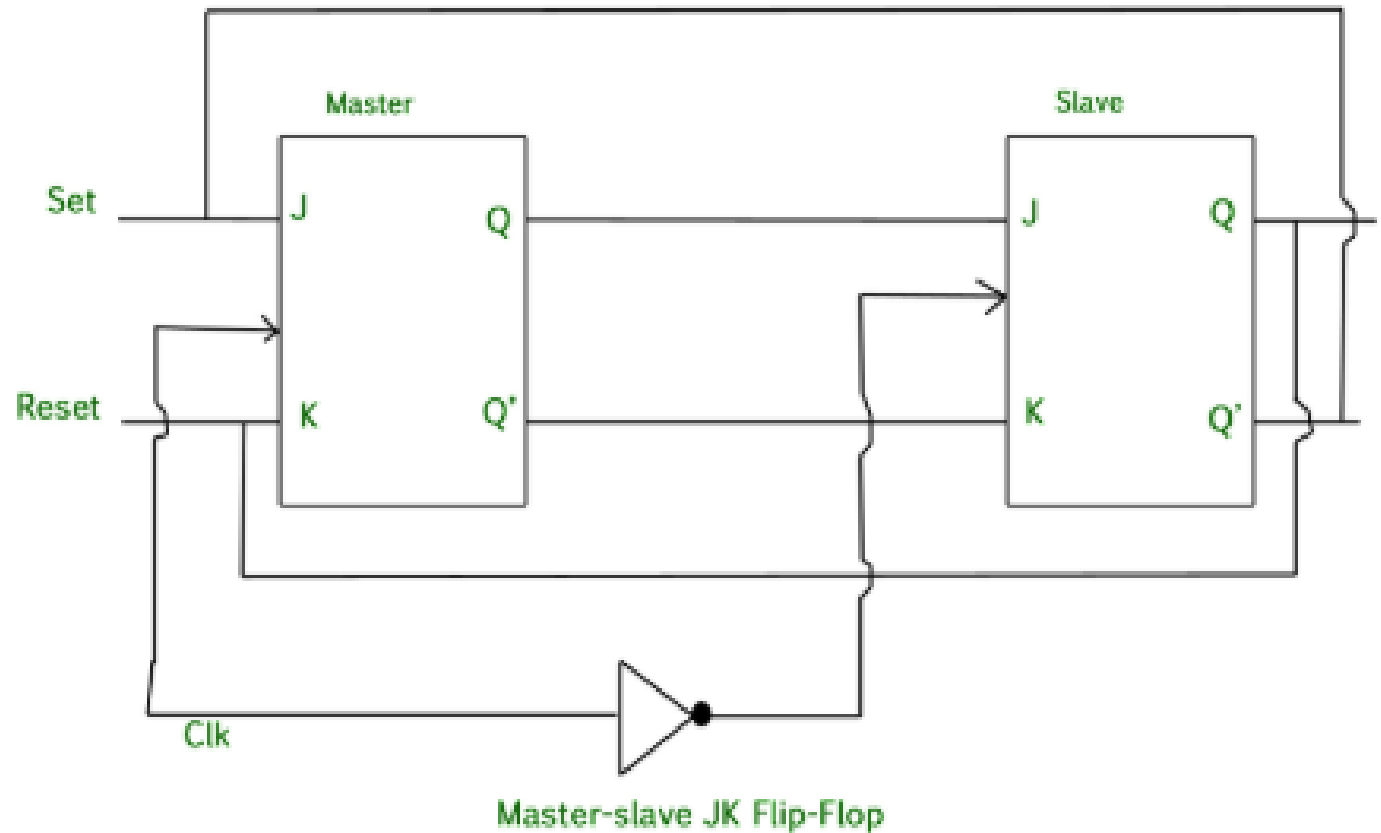
## How to Fix It

- Use Edge-Triggered Flip-Flops – Only respond to rising or falling edge, not level
- Use Master-Slave Flip-Flops – Two-stage design to avoid multiple toggles

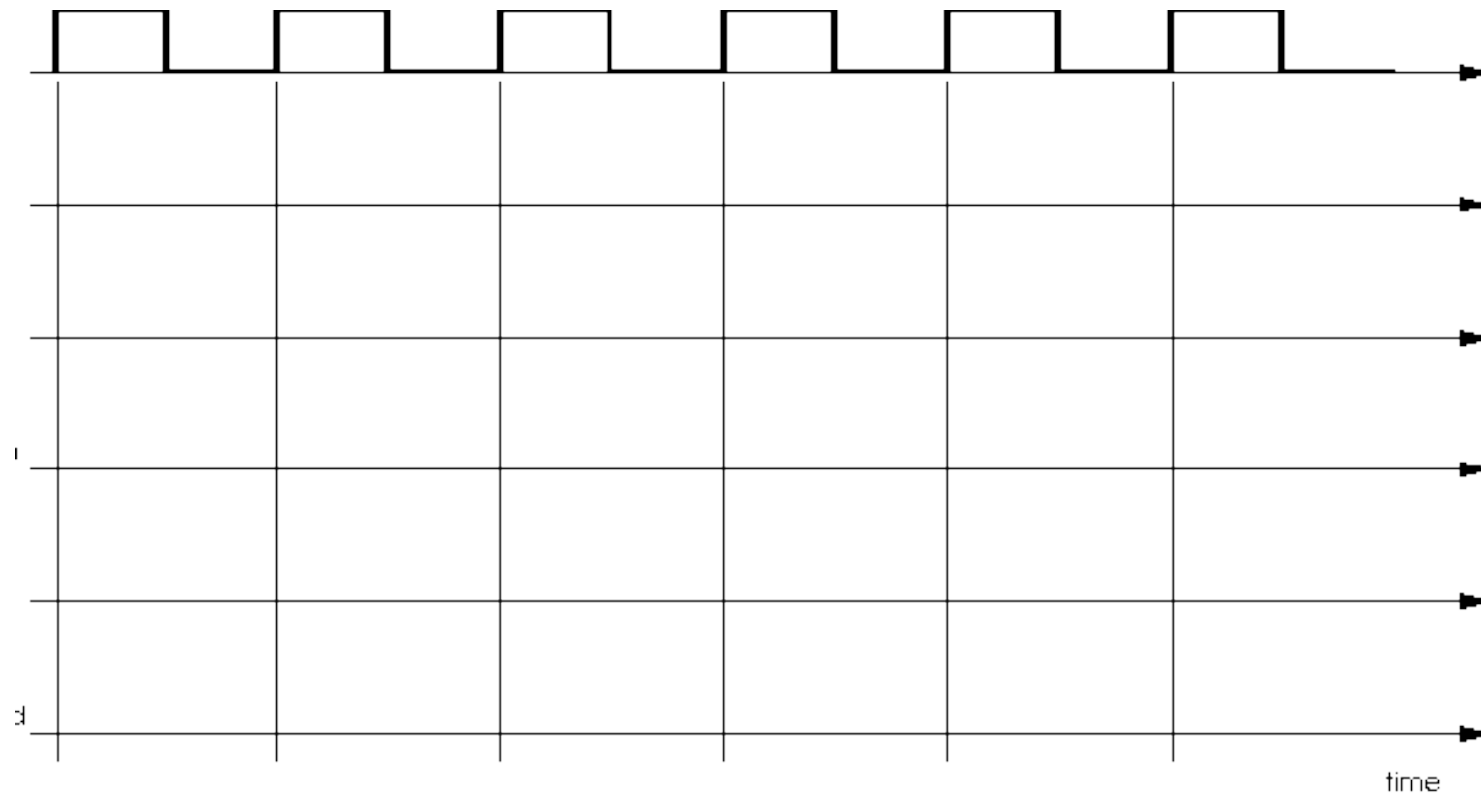
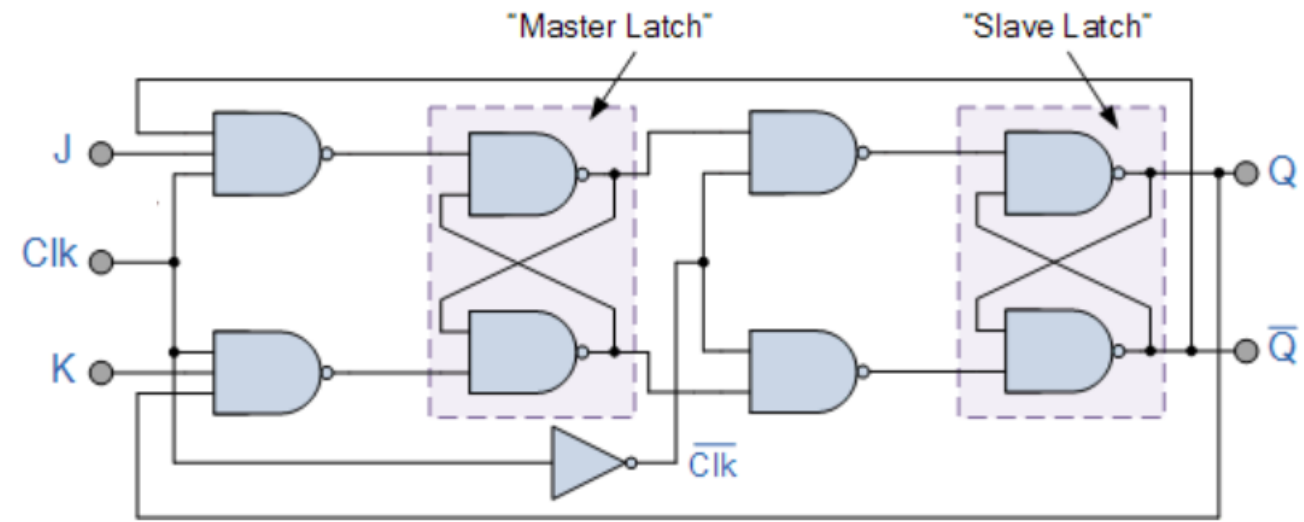


# Master-Slave JK flipflops

They are controlled by opposite phases of the clock, so they never toggle at the same time. This ensures that changes happen only once per clock cycle, avoiding race-around.



# Master-Slave JK



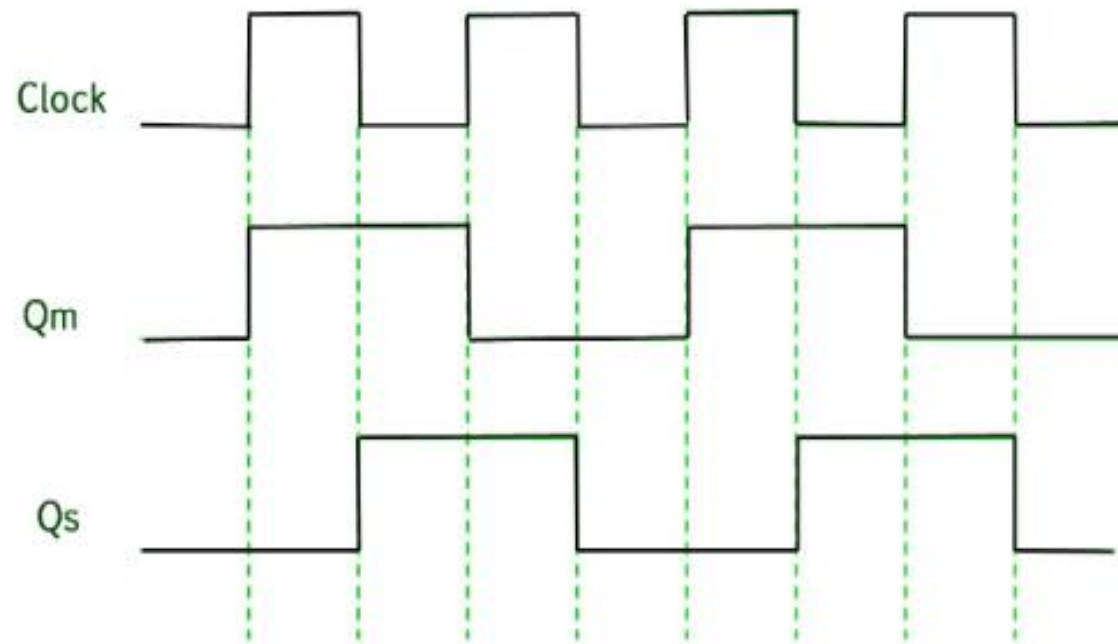
# Timing logic in Master-Slave JK flipflops

1. When the Clock pulse is high the output of master is high and remains high till the clock is low because the state is stored.
2. Now the output of master becomes low when the clock pulse becomes high again and remains low until the clock becomes high again.
3. Thus toggling takes place for a clock cycle.
4. When the clock pulse is high, the master is operational but not the slave thus the output of the slave remains low till the clock remains high.
5. When the clock is low, the slave becomes operational and remains high until the clock again becomes low.

# Timing logic in Master-Slave JK flip-flops

6. Toggling takes place during the whole process since the output is changing once in a cycle.

This makes the Master-Slave J-K flip flop a Synchronous device as it only passes data with the timing of the clock signal.



# Timing logic in Master-Slave JK flip-flops

When Clock = 1 (High):

- Master is active, takes inputs J and K
- Slave is inactive, holds its state

When Clock = 0 (Low):

- Master is inactive, holds its state
- Slave is active, takes the master's output as input

So the output changes only on the falling edge of the clock.

| J | K | Q (next)  |
|---|---|-----------|
| 0 | 0 | No change |
| 0 | 1 | Reset     |
| 1 | 0 | Set       |
| 1 | 1 | Toggle    |

# Master-Slave D flip-flops

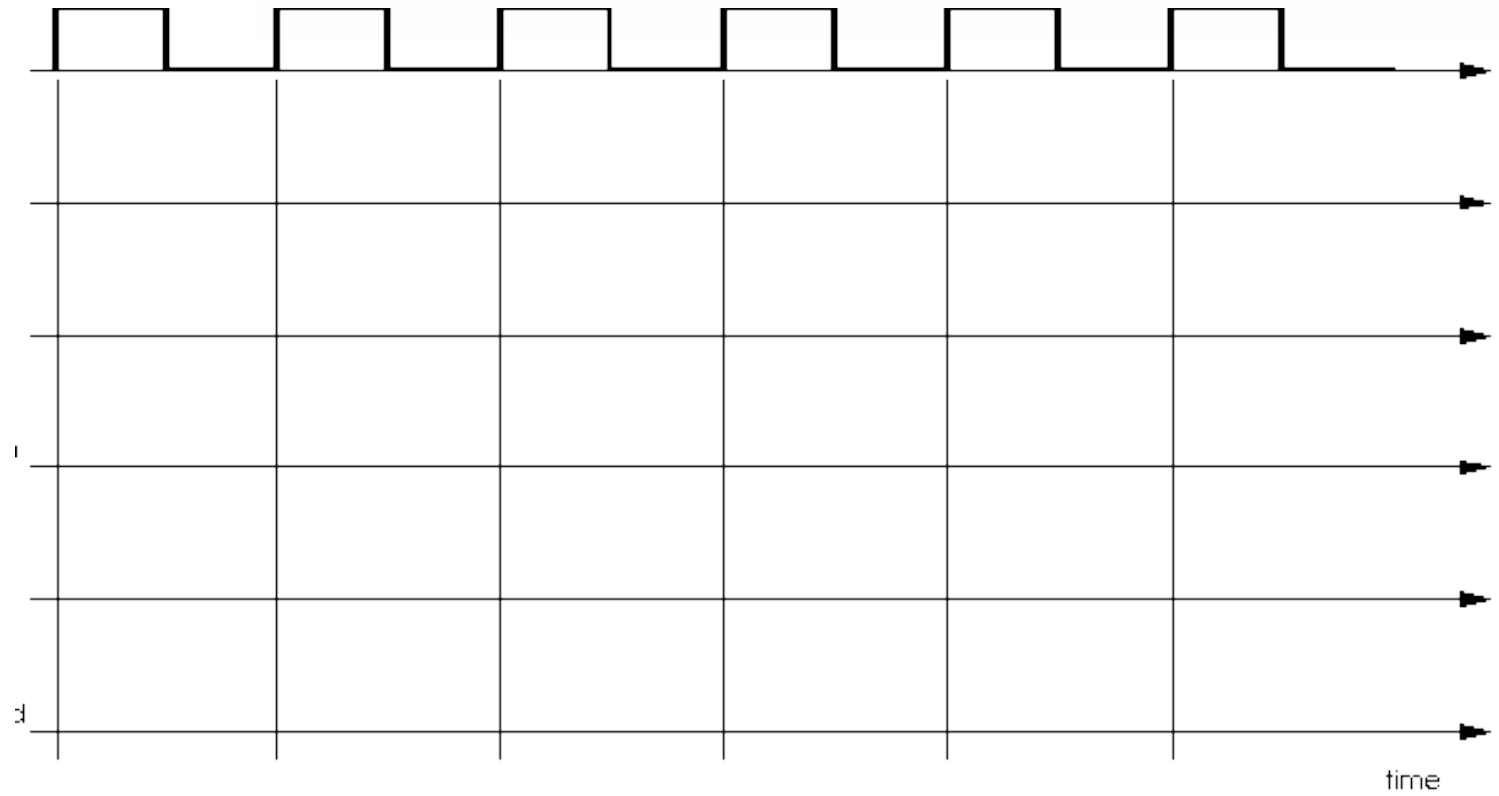
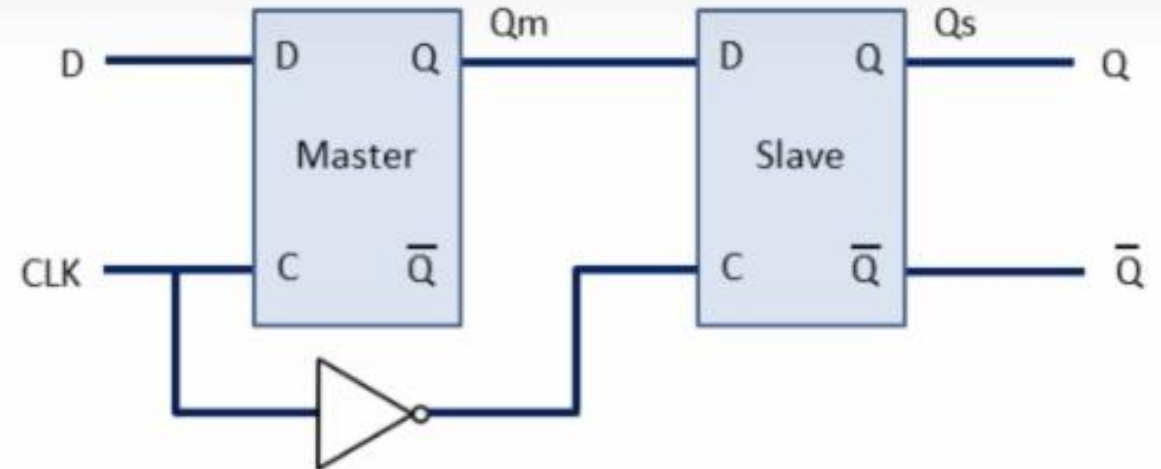
This is a negative edge triggered configuration. Both latches are initially at reset.

$Q_m$  – output waveform of master

$Q_s$  – output waveform of slave

$Q_p$  – output of positive edge triggered flip-flop

$Q_n$  – negative edge triggered D flip-flop



# Master-Slave D flip-flops

## GLITCH

A glitch in a flip-flop is an undesirable, temporary transition or spike in the output signal (0 to 1 to 0 or 1 to 0 to 1) before the output settles to its intended state. It acts as a transient fault, often caused by race conditions, unequal signal path delays, or asynchronous inputs arriving near the active clock edge, which can lead to unexpected state changes.

# Comparison of Flip flops

| Flip-Flop | Inputs | Characteristic Behavior   | Toggling |
|-----------|--------|---------------------------|----------|
| SR        | S, R   | Set, Reset, Hold, Invalid | No       |
| JK        | J, K   | Set, Reset, Hold, Toggle  | Yes      |
| D         | D      | Output follows input      | No       |
| T         | T      | Toggle when $T = 1$       | Yes      |



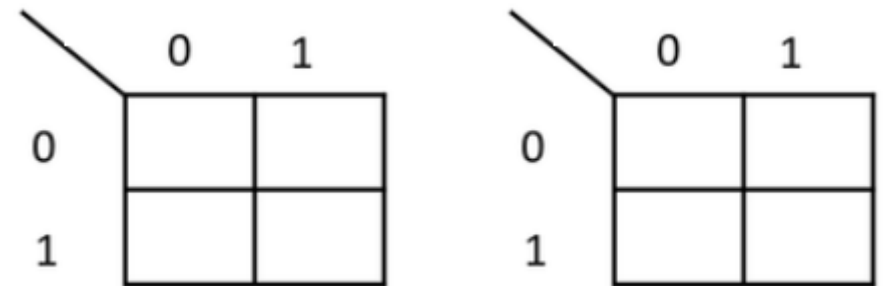
# Conversions of Flip flops

## 1. SR flipflop to D flipflop

| D | $Q_n$ | $Q_{n+1}$ |
|---|-------|-----------|
| 0 | 0     | 0         |
| 0 | 1     | 0         |
| 1 | 0     | 1         |
| 1 | 1     | 1         |

Excitation table of SR

| $Q_n$ | $Q_{n+1}$ | S | R |
|-------|-----------|---|---|
| 0     | 0         | 0 | 0 |
| 0     | 1         | 1 | 0 |
| 1     | 0         | 0 | 1 |
| 1     | 1         | 0 | 0 |



# Conversions of Flip flops

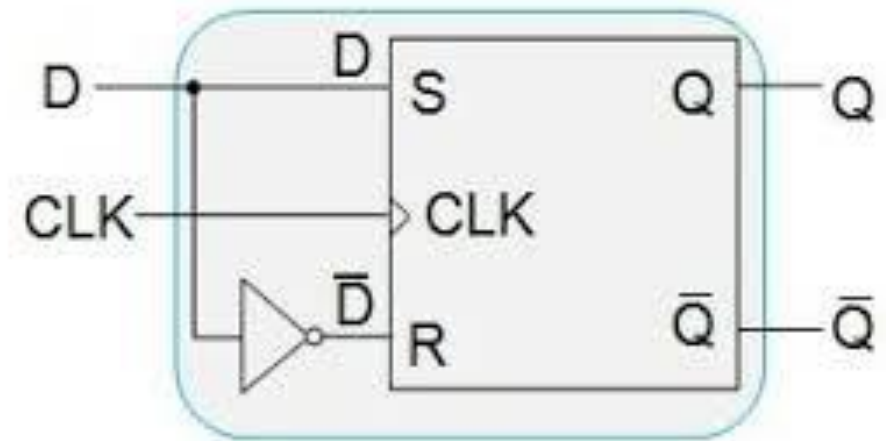
## 1. SR flipflop to D flipflop

| D | $Q_n$ | $Q_{n+1}$ | S | R |
|---|-------|-----------|---|---|
| 0 | 0     | 0         | 0 | x |
| 0 | 1     | 0         | x | 1 |
| 1 | 0     | 1         | 1 | x |
| 1 | 1     | 1         | x | 0 |

$$S = D$$
$$R = D'$$

Excitation table of SR

| $Q_n$ | $Q_{n+1}$ | S | R |
|-------|-----------|---|---|
| 0     | 0         | 0 | 0 |
| 0     | 1         | 1 | 0 |
| 1     | 0         | 0 | 1 |
| 1     | 1         | 0 | 0 |



# Conversions of Flip flops

## 2. T flipflop to JK flipflop

Excitation table of T flipflop

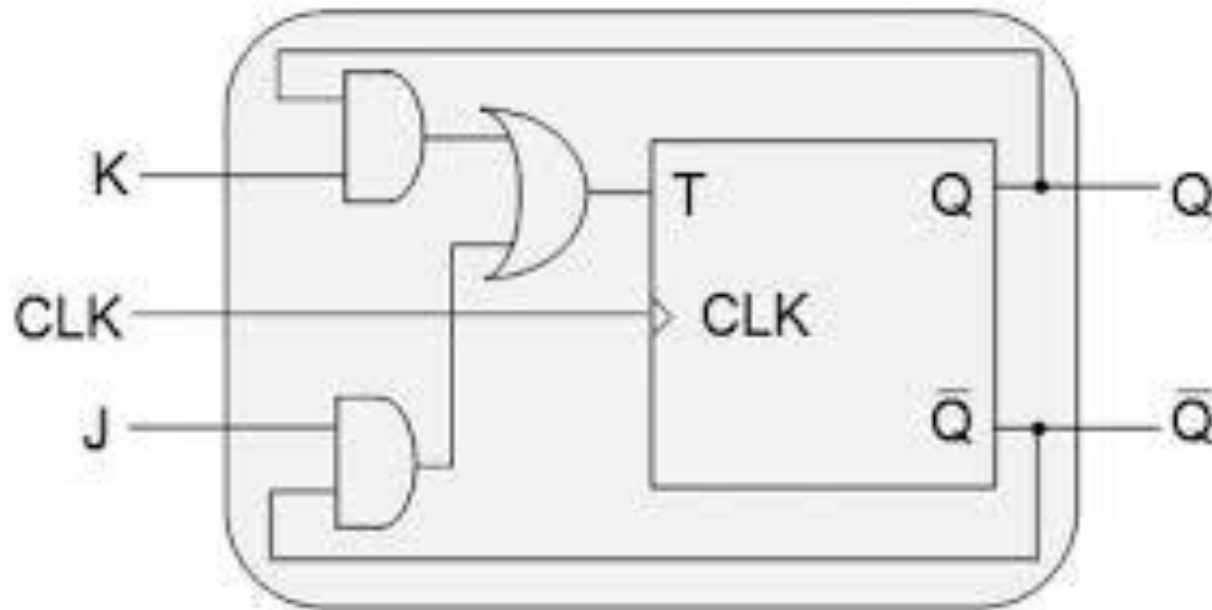
| Q(current) | Q(next) | T |
|------------|---------|---|
| 0          | 0       | 0 |
| 0          | 1       | 1 |
| 1          | 0       | 1 |
| 1          | 1       | 0 |

| J | K | $Q_N$ | $Q_{N+1}$ |
|---|---|-------|-----------|
| 0 | 0 | 0     | 0         |
| 0 | 0 | 1     | 1         |
| 0 | 1 | 0     | 0         |
| 0 | 1 | 1     | 0         |
| 1 | 0 | 0     | 1         |
| 1 | 0 | 1     | 1         |
| 1 | 1 | 0     | 1         |
| 1 | 1 | 1     | 0         |

|   |    |    |    |    |
|---|----|----|----|----|
|   | 00 | 01 | 11 | 10 |
| 0 |    |    |    |    |
| 1 |    |    |    |    |

# Conversions of Flip flops

## 2. T flipflop to JK flipflop



# Conversions of Flip flops

## 3. JK flipflop to SR flipflop

Excitation table of JK flipflop

| Q(current) | Q(next) | J | K |
|------------|---------|---|---|
| 0          | 0       | 0 | X |
| 0          | 1       | 1 | X |
| 1          | 0       | X | 1 |
| 1          | 1       | X | 0 |

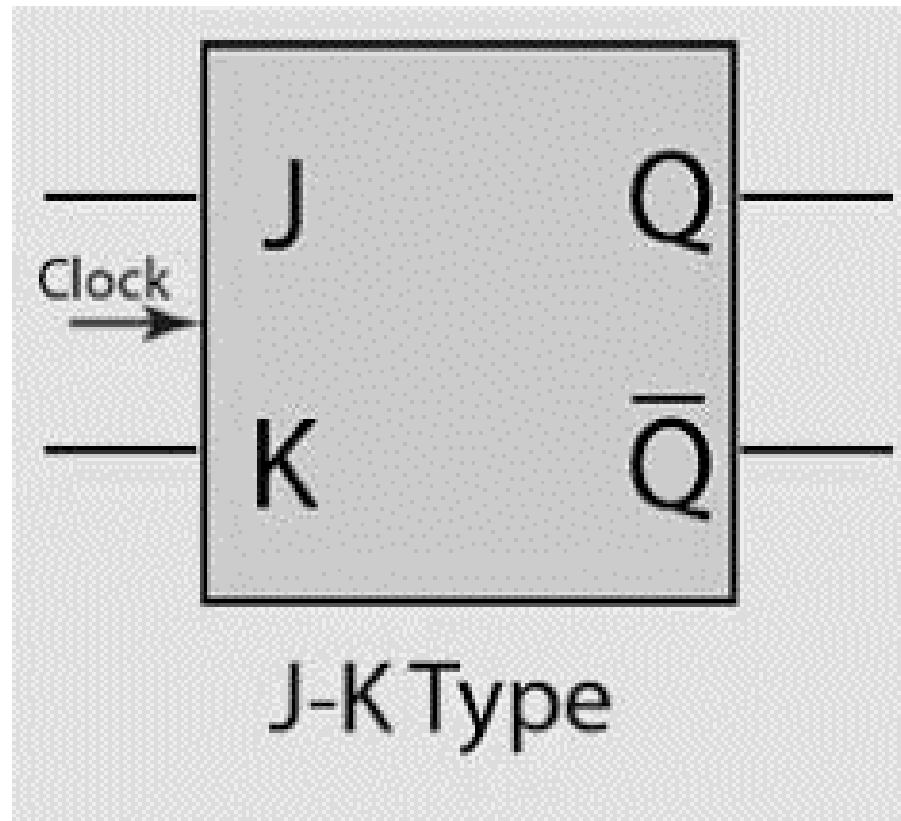
| Input s |   | Present State  | Next State       | Flip Flop Input |   |
|---------|---|----------------|------------------|-----------------|---|
| S       | R | Q <sub>n</sub> | Q <sub>n+1</sub> | J               | K |
| 0       | 0 | 0              | 0                |                 |   |
| 0       | 0 | 1              | 1                |                 |   |
| 0       | 1 | 0              | 0                |                 |   |
| 0       | 1 | 1              | 0                |                 |   |
| 1       | 0 | 0              | 1                |                 |   |
| 1       | 0 | 1              | 1                |                 |   |
| 1       | 1 | 0              | X                |                 |   |
| 1       | 1 | 1              | X                |                 |   |

|   |    |    |    |    |
|---|----|----|----|----|
|   | 00 | 01 | 11 | 10 |
| 0 |    |    |    |    |
| 1 |    |    |    |    |

|   |    |    |    |    |
|---|----|----|----|----|
|   | 00 | 01 | 11 | 10 |
| 0 |    |    |    |    |
| 1 |    |    |    |    |

# Conversions of Flip flops

## 3. JK flipflop to SR flipflop



# Level and Edge Trigger



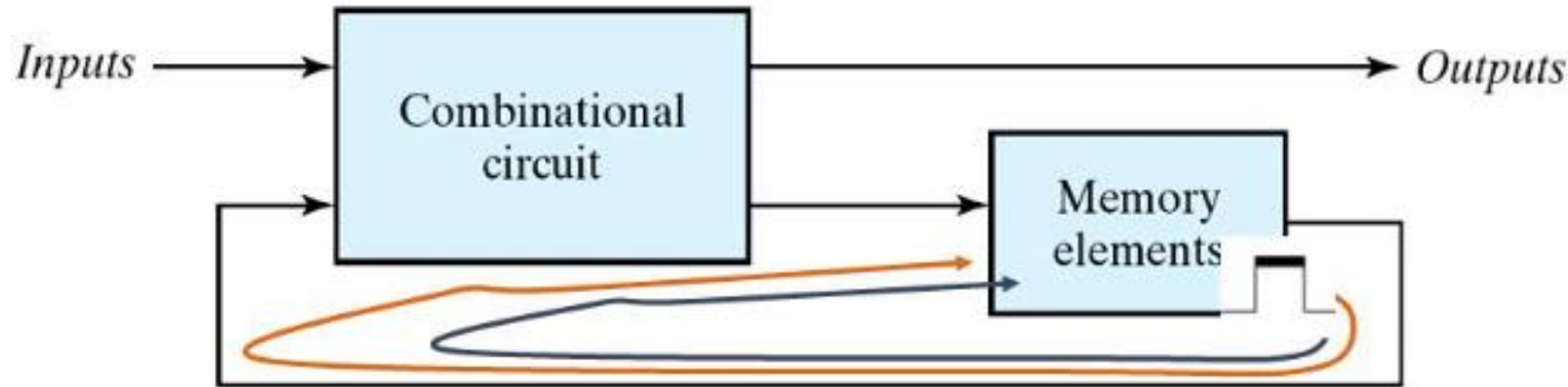
(a) Response to positive level



(b) Positive-edge response



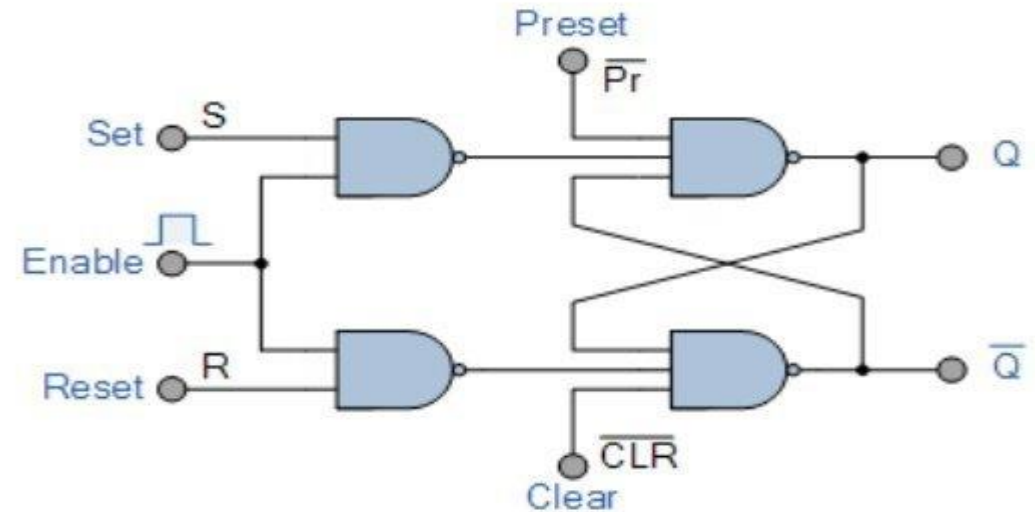
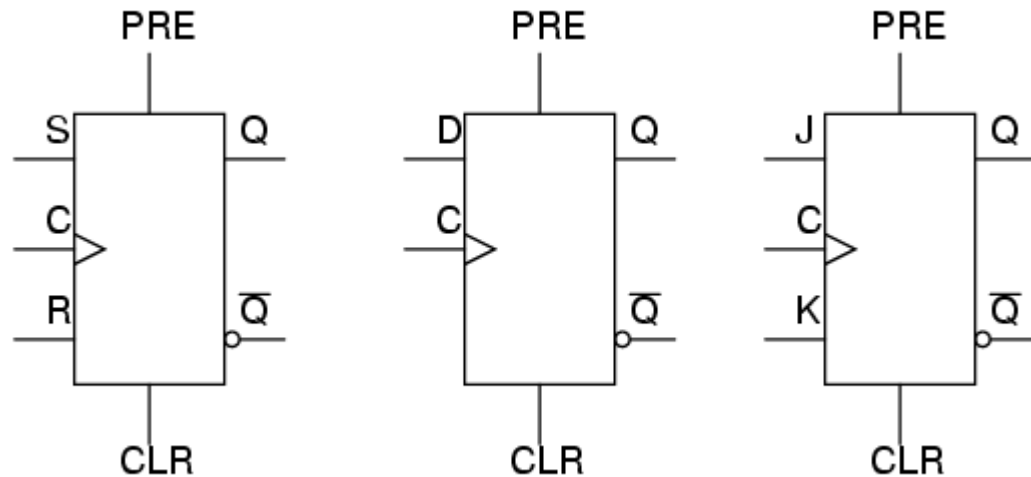
(c) Negative-edge response



- If level-triggered flip-flops are used
  - the feedback path may cause instability problem (since the time interval of logic-1 is too long)
  - multiple transitions might happen during logic-1 level
- Edge-triggered flip-flops
  - the state transition happens only at the edge
  - eliminate the multiple-transition problem



# Asynchronous Pins (Preset & Clear)



- *Asynchronous* inputs on a flip-flop have control over the outputs (Q and not-Q) regardless of clock input status.
- These inputs are called the *preset* (PRE) and *clear* (CLR). The preset input drives the flip-flop to a set state while the clear input drives it to a reset state.
- It is possible to drive the outputs of a J-K flip-flop to an invalid condition using the asynchronous inputs, because all feedback within the multivibrator circuit is overridden.

Thank you!